Clustering Algorithms

Affinity Propagation · Mean Shift · Spectral Clustering
DBSCAN · OPTICS · BIRCH

By Muthuraman P

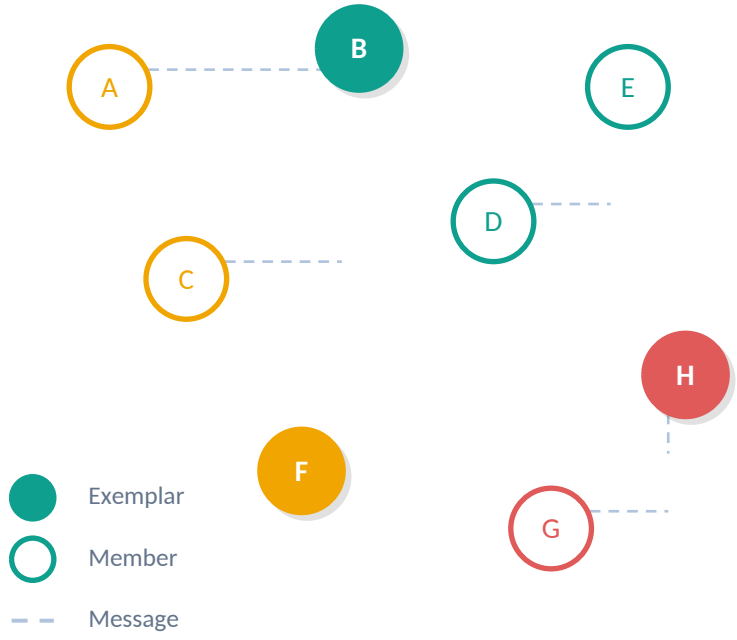
Affinity Propagation

Definition & Diagram

What is it?

Affinity Propagation is a clustering algorithm that identifies exemplars (representative data points) among data points by passing messages between pairs of samples. Unlike K-Means, it does not require the number of clusters to be specified beforehand. The number of clusters is determined by the algorithm based on two key messages: responsibility and availability.

Visual Diagram



Affinity Propagation

Pros & Cons

✔ Advantages

- No need to specify the number of clusters in advance
- Automatically identifies exemplars (representative points)
- Works well with non-convex clusters and irregular shapes
- Can handle datasets with varying cluster sizes
- Produces consistent results (deterministic algorithm)

✘ Disadvantages

- Computationally expensive: $O(n^2)$ time and memory complexity
- Slow on large datasets due to high memory requirements
- Sensitive to the 'preference' and 'damping' hyperparameters
- May produce too many or too few clusters if parameters are poor
- Not suitable for very large datasets (100k+ samples)

Affinity Propagation

Python Code

```python
# Python Implementation

from sklearn.cluster import AffinityPropagation
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

# Generate sample data
X, _ = make_blobs(n_samples=300, centers=4, random_state=42)

# Create and fit the model
model = AffinityPropagation(damping=0.9, preference=-200)
model.fit(X)

# Get cluster labels and exemplar indices
labels = model.labels_
exemplars = model.cluster_centers_indices_
n_clusters = len(exemplars)
print(f'Estimated clusters: {n_clusters}')
```

- damping controls convergence
- preference sets cluster count
- cluster_centers_indices_ = exemplars

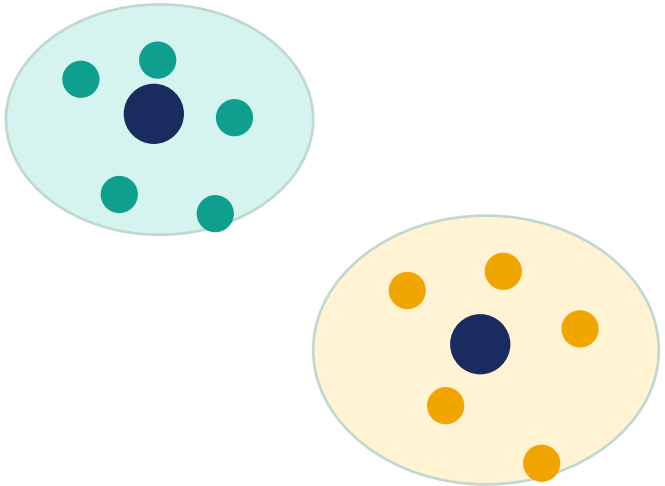
Mean Shift

Definition & Diagram

What is it?

Mean Shift is a non-parametric, density-based clustering algorithm. It works by iteratively shifting each data point toward the region of highest density (the mean of points within a bandwidth window). Points converge to local density maxima, which become cluster centers. It does not require specifying the number of clusters and can find arbitrary-shaped clusters.

Visual Diagram



← *Mean Shift Direction*

● = Cluster Center

Mean Shift

Pros & Cons

✔ Advantages

- No need to pre-specify the number of clusters
- Finds arbitrary-shaped clusters, not just spherical
- Robust to outliers in low-dimensional data
- Model-free — no assumptions about cluster shape
- Automatically determines cluster count via bandwidth

✘ Disadvantages

- Requires bandwidth (kernel size) selection which impacts results
- Computationally expensive: $O(n^2)$ per iteration
- Slow convergence on large datasets
- Struggles in high-dimensional spaces (curse of dimensionality)
- Not suitable for datasets with large variation in cluster density

Mean Shift

Python Code

```python
# Python Implementation

from sklearn.cluster import MeanShift, estimate_bandwidth
from sklearn.datasets import make_blobs

# Generate data
X, _ = make_blobs(n_samples=300, centers=3, random_state=42)

# Estimate bandwidth automatically
bandwidth = estimate_bandwidth(X, quantile=0.2, n_samples=200)

# Create and fit model
model = MeanShift(bandwidth=bandwidth, bin_seeding=True)
model.fit(X)

labels = model.labels_
centers = model.cluster_centers_
n_clusters = len(set(labels))
print(f'Clusters found: {n_clusters}')
```

- estimate_bandwidth() auto-tunes
- bin_seeding speeds computation
- cluster_centers_ = density peaks

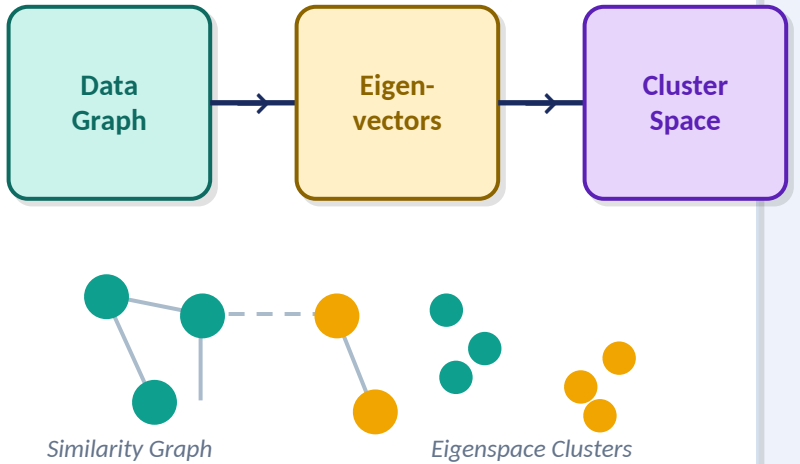
Spectral Clustering

Definition & Diagram

What is it?

Spectral Clustering uses the eigenvalues (spectrum) of a similarity matrix to perform dimensionality reduction before clustering. It constructs a graph where data points are nodes and edges encode similarity, then uses the graph Laplacian's eigenvectors to embed data into a lower-dimensional space where clusters become more separable, and applies K-Means in that space.

Visual Diagram



Spectral Clustering

Pros & Cons

✔ Advantages

- Can detect clusters with complex, non-convex shapes
- Effective on graph-structured or manifold data
- Works well when cluster boundaries are unclear in input space
- Flexible: different affinity/kernel functions can be used
- Theoretically well-grounded (spectral graph theory)

✘ Disadvantages

- Must specify the number of clusters (n_clusters) upfront
- Computationally expensive for large datasets: $O(n^3)$
- Memory-intensive due to the full affinity matrix
- Sensitive to the choice of affinity/kernel hyperparameters
- Does not scale well beyond ~10,000 data points

Spectral Clustering

Python Code

```python
# Python Implementation

from sklearn.cluster import SpectralClustering
from sklearn.datasets import make_moons
import numpy as np

# Make non-linearly separable data (moons)
X, y = make_moons(n_samples=300, noise=0.05, random_state=42)

# Spectral Clustering
model = SpectralClustering(
    n_clusters=2,
    affinity='rbf',   # Radial Basis Function kernel
    gamma=1.0,
    random_state=42
)
labels = model.fit_predict(X)
print('Unique clusters:', np.unique(labels))
```

affinity='rbf' handles non-linear data

gamma controls kernel width

fit_predict() in one step

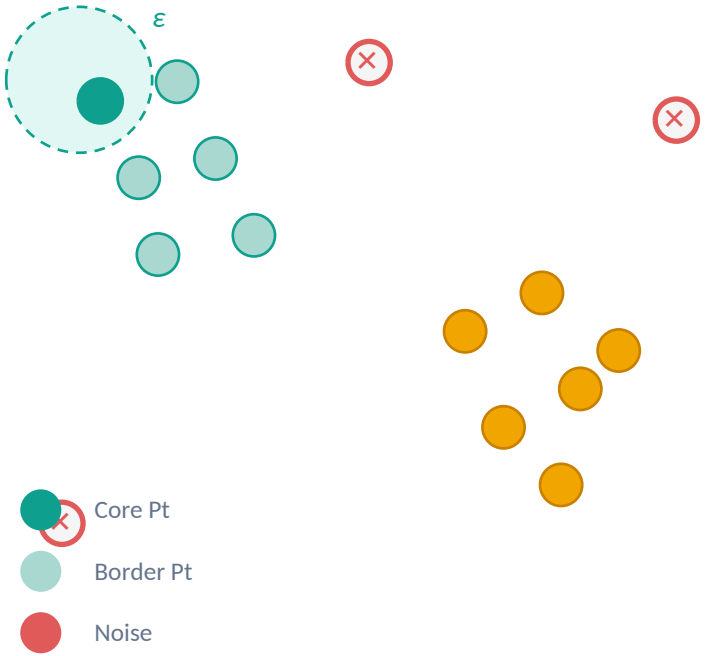
DBSCAN

Definition & Diagram

What is it?

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) groups points that are closely packed together while marking points in low-density regions as outliers. It defines clusters based on two parameters: epsilon (ε) — the neighborhood radius — and min_samples — the minimum points needed within ε to form a dense region. Points are classified as core, border, or noise.

Visual Diagram



DBSCAN

Pros & Cons

✔ Advantages

- No need to specify the number of clusters in advance
- Can find arbitrarily shaped clusters
- Naturally identifies and handles noise/outliers
- Robust to outliers unlike K-Means
- Efficient on large spatial datasets with indexing

✘ Disadvantages

- Requires careful tuning of epsilon and min_samples
- Struggles with clusters of varying density
- Performance degrades in high-dimensional spaces
- Not fully deterministic with border points
- Difficult to cluster datasets with very different densities

DBSCAN

Python Code

```python
# Python Implementation

from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons
import numpy as np

X, _ = make_moons(n_samples=300, noise=0.05, random_state=42)

# Fit DBSCAN
model = DBSCAN(
    eps=0.3,          # Neighborhood radius
    min_samples=5     # Min points to form core
)
labels = model.fit_predict(X)

n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
n_noise = list(labels).count(-1)
print(f'Clusters: {n_clusters}, Noise: {n_noise}')
```

label -1 = noise point

eps defines neighborhood size

min_samples controls density

OPTICS

Definition & Diagram

What is it?

OPTICS (Ordering Points To Identify the Clustering Structure) is an extension of DBSCAN that addresses its limitation of varying cluster densities. It creates an ordered list of data points with reachability distances, producing a reachability plot. Valleys in this plot correspond to clusters, enabling detection of clusters at multiple density levels without a fixed epsilon.

Visual Diagram

↓ *valleys = clusters*

OPTICS

Pros & Cons

✔ Advantages

- Handles clusters of varying densities better than DBSCAN
- No need to specify epsilon — uses max_eps instead
- Produces a reachability plot for visual cluster analysis
- Can identify clusters at multiple density levels (hierarchical)
- Robust noise/outlier detection

✘ Disadvantages

- More computationally expensive than DBSCAN
- Reachability plot can be difficult to interpret automatically
- Extracting exact clusters still requires parameter choices
- Slower on very large datasets
- Not widely supported in all ML frameworks

OPTICS

Python Code

```
# Python Implementation

from sklearn.cluster import OPTICS
from sklearn.datasets import make_blobs
import numpy as np

X, _ = make_blobs(n_samples=300, centers=3, random_state=42)

# Fit OPTICS model
model = OPTICS(
    min_samples=10,
    xi=0.05,          # Cluster steepness threshold
    min_cluster_size=0.1
)
model.fit(X)

labels = model.labels_
reach_dist = model.reachability_[model.ordering_]
print('Clusters found:', len(set(labels)) - (1 if -1 in labels else 0))
```

- xi controls valley detection
- reachability_ for visual plot
- ordering_ = traversal order

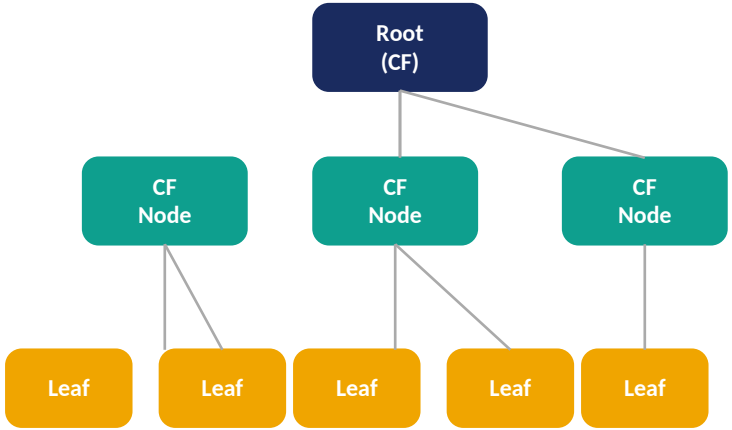
BIRCH

Definition & Diagram

What is it?

BIRCH (Balanced Iterative Reducing and Clustering using Hierarchies) is designed for large datasets. It builds a Clustering Feature Tree (CF Tree), a compact height-balanced tree that summarizes the dataset. Each node stores a Clustering Feature (CF) — a tuple of (n, LS, SS) representing count, linear sum, and squared sum — enabling incremental, single-pass clustering with limited memory.

Visual Diagram



CF = (N, LS, SS)

N=count LS=linear sum SS=sq. sum

BIRCH

Pros & Cons

✔ Advantages

- Highly scalable — designed for very large datasets
- Single-pass algorithm: efficient memory and time usage
- Incrementally updates clusters as new data arrives
- Handles noise/outliers through the CF tree structure
- Much faster than K-Means on large datasets

✘ Disadvantages

- Works best only with numeric, continuous features
- Cluster quality depends heavily on threshold parameter (T)
- May not handle non-spherical clusters well
- Requires a second-pass clustering algorithm for best results
- Less effective in very high-dimensional spaces

BIRCH

Python Code

```
# Python Implementation

from sklearn.cluster import Birch
from sklearn.datasets import make_blobs
import numpy as np

# Large dataset simulation
X, _ = make_blobs(n_samples=10000, centers=5, random_state=42)

# Fit BIRCH model
model = Birch(
    threshold=0.5,    # Max radius of each sub-cluster
    branching_factor=50,
    n_clusters=5      # Final cluster count
)
labels = model.fit_predict(X)

print('Unique clusters:', np.unique(labels))
print('Subcluster count:', model.subcluster_labels_.shape)
```

threshold controls CF node size

branching_factor = tree width

n_clusters=None returns subclusters